

SATGUARD: Concealing Endless and Bursty Packet Losses in LEO Satellite Networks for Delay-Sensitive Web Applications

Jihao Li
lijh19@mails.tsinghua.edu.cn
Tsinghua University
Beijing, China

Hewu Li
lihewu@cernet.edu.cn
Tsinghua University
Zhongguancun Laboratory
Beijing, China

Zeqi Lai
zeqilai@tsinghua.edu.cn
Tsinghua University
Zhongguancun Laboratory
Beijing, China

Qian Wu
wuqian@cernet.edu.cn
Tsinghua University
Zhongguancun Laboratory
Beijing, China

Yijie Liu
liuyj23@mails.tsinghua.edu.cn
Tsinghua University
Beijing, China

Qi Zhang
zhangqi@zgclab.edu.cn
Zhongguancun Laboratory
Beijing, China

Yuanjie Li
yuanjiel@tsinghua.edu.cn
Tsinghua University
Zhongguancun Laboratory
Beijing, China

Jun Liu
juneliu@tsinghua.edu.cn
Tsinghua University
Zhongguancun Laboratory
Beijing, China

ABSTRACT

Delay-sensitive Web services are crucial applications in emerging low-earth orbit (LEO) satellite networks (LSNs). However, our real-world measurement study based on SpaceX's Starlink, the most widely used commercial LSN today, reveals that the endless and bursty packet losses over unstable LEO satellite links impose significant challenges on guaranteeing the quality of experience (QoE) of Web applications. We propose SATGUARD, a distributed in-orbit loss recovery mechanism that can reduce user-perceived delay by completely concealing packet losses in the unstable and lossy LSN environment from endpoints. Specifically, SATGUARD adopts a series of techniques to: (i) correctly migrate on-board packet buffer to support link-local retransmission under LEO dynamics; (ii) efficiently detect packet losses on satellite links; and (iii) ensure packet ordering for endpoints. We implement a SATGUARD prototype, and conduct extensive trace-driven evaluations guided by public constellation information and real-world measurements. Our experiments demonstrate that, in comparison with other state-of-the-art approaches, SATGUARD can significantly improve Web-based QoE, by reducing: (i) up to 48.3% of page load time for Web browsing; and (ii) up to 57.4% end-to-end communication delay for WebRTC.

CCS CONCEPTS

• Information systems → Web applications; • Networks → Network reliability; • General and reference → Measurement.

KEYWORDS

LEO Satellite Networks; Loss Recovery; Web Application

ACM Reference Format:

Jihao Li, Hewu Li, Zeqi Lai, Qian Wu, Yijie Liu, Qi Zhang, Yuanjie Li, and Jun Liu. 2024. SATGUARD: Concealing Endless and Bursty Packet Losses in LEO Satellite Networks for Delay-Sensitive Web Applications. In *Proceedings of the ACM Web Conference 2024 (WWW '24)*, May 13–17, 2024, Singapore.



This work is licensed under a Creative Commons Attribution International 4.0 License.

WWW '24, May 13–17, 2024, Singapore, Singapore
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0171-9/24/05.
<https://doi.org/10.1145/3589334.3645639>

Singapore. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3589334.3645639>

1 INTRODUCTION

We are experiencing two major changes in today's Internet. The delay requirements of upper-layer applications are increasing, driven by a range of new technologies [32, 38]. Simultaneously, the underlying network architecture is evolving from the land to outer space, stimulated by the recent low-earth orbit (LEO) satellite constellations. Big companies, such as SpaceX [57], Telesat [13], and Amazon [5], are planning and deploying satellite constellations (e.g., Starlink and Kuiper) for global Internet access. These constellations, consisting of a large number of broadband satellites equipped with laser inter-satellites links (ISLs) [12, 64], ground-satellite links (GSLs) for communicating with ground stations, and user-satellite links (USLs) for connecting to users, extend the boundary of today's Internet and construct LEO satellite networks (LSNs) to provide low-delay and high-speed Internet services ubiquitously [29, 35].

Web-based applications should be important scenarios in the upcoming era of satellite Internet. Representative Web applications, such as Web browsing and Web-based real-time communication (e.g., WebRTC [27]) are expected to provide a good quality of experience (QoE) for LSN users. However, our real-world measurements on Web applications in the operational Starlink LSN plot a gloomy picture: while the low orbit altitude guarantees low baseline propagation delay for LSN users, the inherently unstable and lossy satellite links impose significant challenges on sustaining good Web QoE. Since LEO satellites continuously move at high velocities relative to the earth's surface, such inherent LEO dynamics can result in *endless and burst* packet losses for transport endpoints. Although the existing sender-side reliability control used in *status quo* Web applications can recover lost packets, retransmission from the sender inevitably involves extra delay and high jitter which can further hinder user-perceived Web experience (e.g., prolonging Web page load time and end-to-end communication delay).

Existing approaches accomplishing fast loss recovery (and thus reduce the user-perceived delay) can be classified into two major categories. On the one hand, many efforts [20, 30, 46, 66] proposed to use network coding (e.g., forward error correction, FEC) to inject redundant information in data packets on the source, and recover

the lost packets on the receiver without the need of sender-side retransmission. However, due to the highly dynamic packet loss rate, a lower redundancy coding information may lead to recovery failure while a higher redundancy one requires much additional bandwidth, which is scarce in LSNs. Although recent studies have used machine learning technologies to predict packet loss rate [30, 39], the used datasets can not reflect unique network behaviors in LSN scenarios (e.g., handovers in satellite systems and signal attenuation caused by objects blocking the view of satellite terminals). On the other hand, other works proposed link-local retransmission (LL-ReTx) to buffer packets on intermediate network nodes and re-send lost packets locally. This idea is mainly implemented in either (i) the transport layer [4, 8, 36, 41, 50] (i.e., intermediate nodes have full transport-layer functions and maintain information for each flow, thus splitting the end-to-end connection into segments) or (ii) the link layer [24, 33, 51] (i.e., only buffering packets or frames at the network interface and retransmitting them when any loss is detected). However, due to the high LEO dynamics, frequent disconnections and reconstructions of satellite links can disrupt the retransmission process and limit the recovery efficiency.

In this paper, we propose SATGUARD, a novel in-orbit loss recovery mechanism that can be deployed in operational LSNs to effectively improve QoE for Web applications. SATGUARD extends the existing link-local retransmission mechanism to address the unique challenges caused by LEO dynamics and conceal packet losses for endpoints. The core design of SATGUARD incorporates three key techniques: (i) leveraging predictable handover information (e.g., ground-satellite connectivity and handover time) based on the global scheduler of LSN operators to correctly migrate buffered content; (ii) dynamically configuring appropriate parameters of packet loss detection (i.e., timeout for LL-ReTx) for land-based users served by different satellite beams to improve recovery efficiency, and (iii) preserving the order of packets while leaving LSNs without head-of-line blocking, which can reduce the negative impacts of the out-of-order packets generated by LL-ReTx (e.g., increasing jitter buffer greatly) on the upper-layer protocols.

We have implemented a SATGUARD prototype upon Linux, and built an LSN simulation environment based on public constellation information and realistic Starlink data trace. SATGUARD only requires operator-side modifications, and can be deployed in operational LSNs by software upgrades on satellites, ground stations, and satellite terminals, without the need to change the user-side application interface. Our extensive evaluations demonstrate that: in comparison with other existing reliability control approaches, SATGUARD can: (i) reduce 99th percentile end-to-end delay by up to 59.8% and keep higher capacity utilization on average; and (ii) speed up Web browsing time by up to 48.3% and reduce user-perceived communication delay by up to 57.4% for WebRTC sessions.

In summary, main contributions in this paper can be concluded as follows: (i) we conduct a real-world measurement study to identify and analyze the performance issues suffered by representative Web applications in emerging LSNs; (ii) we present SATGUARD, an in-orbit fast loss recovery mechanism which extends existing LL-ReTx mechanisms to accomplish consistent low delay in unstable and lossy LSNs and improve user-perceived QoE; and (iii) we implement a SATGUARD prototype and conduct extensive, real-data-driven evaluations to verify the improvement on Web browsing and RTC.

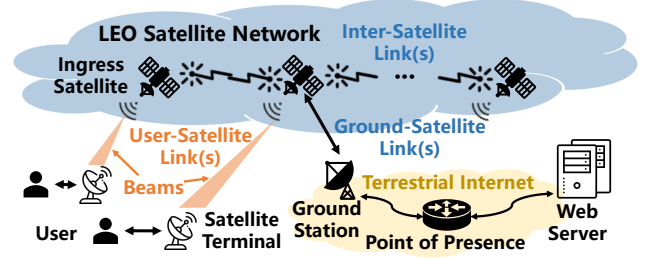


Figure 1: A high-level architecture of today's LSNs.

2 BACKGROUND AND MOTIVATION

2.1 LEO Satellite Networks

Figure 1 plots a typical networking architecture that has been widely used by today's commercial LSNs such as SpaceX's Starlink [63]. At a high level, an LSN contains a *space segment* with a large number of LEO broadband satellites, and a *ground segment* consisting of geo-distributed ground stations (GSeS) and satellite terminals (dish) [54–56, 58]. Satellites can be equipped with laser inter-satellite links (ISLs) for inter-satellite communication, be equipped with radio ground-satellite links (GSLs) for connecting to GSeS, and further be connected to satellite terminals via radio user-satellite links (USLs) consisting of multiple spot-beams. When a user terminal accesses Internet services (e.g., a Web server) through the LSN, the user's request is first forwarded to a ground station through one (i.e., via bent-pipe transparent forwarding [45]) or multiple satellites (i.e., via ISL-based space routing [42]) and then to a Point-of-Presence (PoP) of the Internet. Similarly, the server's response returns to the user terminal via the reverse path.

2.2 Understanding Web Performance in LSNs

To quantitatively understand the user-perceived experience of typical Web applications in emerging LSNs, we conduct a real-world measurement study in Starlink, the currently most popular commercial LSNs with more than 2 million subscribers globally [61].

Experiment setup. With the help of our partners, we deploy two laptops in New York City (NYC), which connect to the Internet via Starlink and Optimum (i.e., a wired broadband service provider) respectively. Further, we measure the QoE of two representative applications: Web browsing and Web Real-Time Communication (WebRTC) via the two different networks. For Web browsing, we use Chrome to visit Alexa Top 200 websites [16]. We then exploit the Browsertime tool [23] to measure: (i) *page load time*, which refers to the time it takes for a Web page to fully load in a browser, and (ii) *speed index*, which measures the visual completeness of a Web page, and lower values indicate faster perceived page loading. For WebRTC, we use Kurento [25] to build sessions between the laptop client in NYC and other clients in Paris (France), Beijing (China), and Davis (U.S.). Based on WebRTC's statistics APIs [62], we measure: (i) end-to-end user-perceived communication delay; and (ii) frames per second (FPS). We use tcpdump to collect packet-level traces of the two applications for further network analysis.

Observations. Figure 2 plots the QoE results of Web browsing in different networks. We observe that in the Starlink network environment, both page load time and speed index are higher than those in the wired network. Specifically, the 60th/80th/90th percentile page

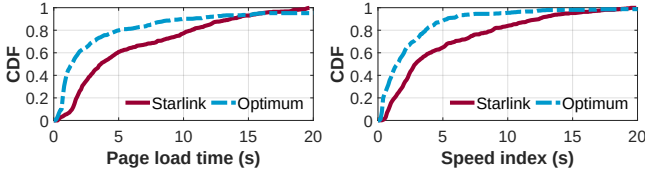


Figure 2: QoE analysis for Alexa Top 200 Web browsing.

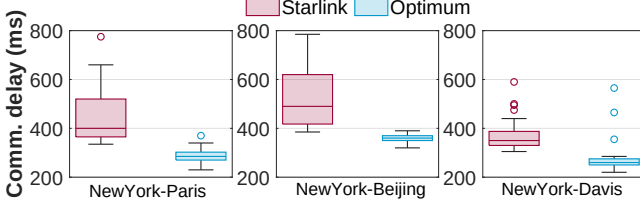


Figure 3: The end-to-end user-perceived communication delay for WebRTC communications.

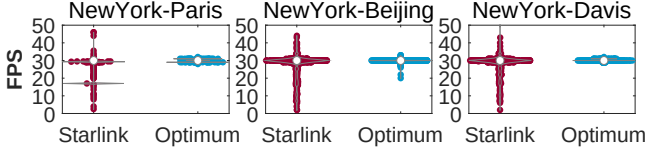


Figure 4: Violin plot of FPS in WebRTC communications.

load time in Starlink are about 245.0%/202.4%/136.3% higher than those in Optimum. Similarly, the 60th/80th/90th percentile speed index in Starlink is about 185.3%/241.6%/219.3% higher, indicating that for the same Web content, LSN users suffer from higher perceived delay and worse browsing experience. Since high page load time can increase the likelihood of users leaving the website [1], optimizing the page load time in LSNs should be an important goal for both network operators and content providers.

In addition, Figure 3 and Figure 4 show the QoE analysis for WebRTC in different networks. We find that for all three sessions associated with users in different locations, they experience higher user-perceived communication delay and fluctuating FPS in Starlink, indicating a worse user experience compared to the wired network. Specifically, as shown in Figure 3, the average communication delay experienced in Starlink is about 57.2%/43.1%/34.5% higher than that in Optimum for the three sessions. As shown in Figure 4, we observe that Optimum achieves a stable FPS with an average value of 30 and a smooth user experience, while the FPS in Starlink is relatively unstable and changes frequently.

Root cause analysis. To uncover the underlying factors that hinder Web QoE, we further conduct an in-depth analysis. Based on the traces collected by tcpdump, we calculate the network-level packet loss rate (PLR) and packet-level one-way delay (OWD), which refers to the duration from the first time the data packet is sent by the sender, to the first time it is received by the receiver. We reveal two LSN-specific factors that *jointly* affect the user-perceived Web QoE.

(i) Endless and bursty packet losses in dynamic LSNs. Figure 5 plots the PLR over time experienced. We find that the bursty PLR ranges from about 3% to 15%. Every 3-5 seconds, a sudden increase in PLR can be clearly observed. Unlike traditional terrestrial Internet where the key network infrastructure is sealed in a protected environment, the core space backbone (*i.e.*, satellite switches or routers) of an LSN is operated and exposed in the *error-prone intermittent space environment*, in which packet losses may occur

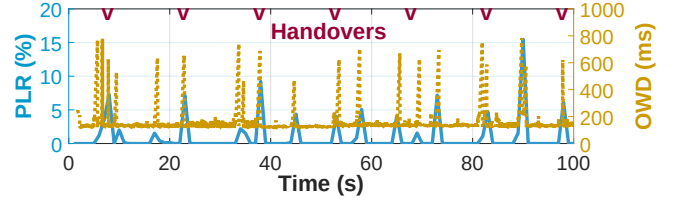


Figure 5: Network-level burst packet loss rate (PLR) and one-way delay (OWD) variation measured in Starlink.

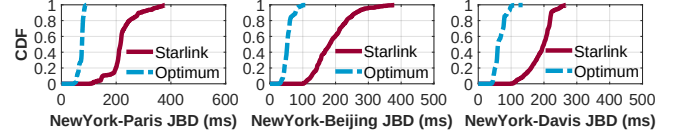


Figure 6: JBD analysis for WebRTC communications.

frequently for both ground-satellite and inter-satellite communication due to a range of factors such as LEO dynamics [7, 34] and electromagnetic interference [31, 44, 45]. For example, recent reports have demystified the Starlink's handover strategy [21, 59, 60], which uses a global scheduler to manage the satellite-terminal connectivity. Handovers occur every 15 seconds between the satellite terminal and its ingress satellite. In Figure 5, we also plot the time point when the handovers occur. We observe that when a handover occurs, it is always accompanied by a noticeable increase in PLR.

(ii) Long tail delay due to sender-based retransmission. To cope with packet losses, existing reliability control typically uses *sender-based retransmission* to guarantee reliable transmission in lossy networks (*e.g.*, TCP retransmission). However, sender-based retransmission can result in *out-of-order packets and long-tail delay* in the lossy LSN environment. We also plot the OWD over time in Figure 5, and we find that high delay bursts are always accompanied by high packet loss bursts. Over the entire session, although the low orbit altitude enables low propagation delay over satellite links, we still observe that the 99th percentile receiver-perceived delay can reach up to 523.3% of the medium value. These high delay variations can seriously affect the Web user experience. On the one hand, a Web page typically consists of a number of objects (*e.g.*, images, texts, and scripts), and the total page load time mainly depends on the last object received. On the other hand, Web-based RTC applications typically use a receiver-side jitter buffer to ensure that the audio and video are played smoothly. The jitter buffer size is determined based on the maximum delay observed by the receiver over a past period of time. Therefore, the high tail delay can increase the jitter buffer delay (JBD) on the receiver, and accordingly amplify the user-perceived end-to-end communication delay. Figure 6 plots the JBD results of the three WebRTC sessions in previous experiments. We find that the burst packet losses and high delay variation indeed increase the JBD for RTC sessions in Starlink, and finally increase the end-to-end communication delay.

2.3 Can Current Fast Recovery Methods Help?

The network community has a long history of research into fast packet recovery, which can be mainly divided into two categories. **Network coding.** One classic technique for fast packet recovery is Forward Errors Correction (FEC) [20, 46, 66]. FEC works by adding

redundant information to the original data packets before transmission. These redundant packets contain additional information that allows the receiver to recover lost or corrupted packets. When a packet is lost or damaged in transit, the receiver uses the redundant packets to reconstruct the missing or corrupted data, thus achieving fast packet recovery without sender-side retransmissions.

However, as shown in Figure 5, packet loss rates in LSNs are highly dynamic. FEC typically applies a fixed level of redundancy to all packets, or simply makes overdue redundancy adjustments based on recently perceived network conditions. Selecting the right level of redundancy can be quite challenging in LSNs. Adding more redundancy to the original data packets improves error recovery capabilities, but significantly increases bandwidth overhead which is a scarce resource in LSNs. Although machine learning technologies can help to train a model that can predict packet loss rates by inputting real-world network measurement data [30, 39], the effectiveness is limited by the quality of the dataset. To our knowledge, there is no large-scale dataset currently that can reflect the unique packet loss behaviors of emerging LSNs described in §2.2.

Link-local retransmission (LL-ReTx). Another existing method for fast packet recovery is to retransmit lost packets directly at intermediate nodes. Many previous efforts deploy buffers at intermediate nodes to realize such LL-ReTx [33, 51], which can greatly shorten the recovery delay and save a lot of bandwidth. However, these solutions are originally designed for static networks such as data center networks where intermediate nodes with stateful buffers are relatively static. The relative positions of intermediate nodes in the LSNs change rapidly, making it challenging to directly apply existing LL-ReTx mechanisms in LSNs, because the upstream node which buffers lost packets may not connect to the downstream node after handovers. Although some methods deploy full transport-layer functions at intermediate nodes to achieve a hop-by-hop transmission [2, 36, 41], which can recover packet losses in dynamic scenarios after new transport connections are built. However, frequent connection reconstruction may not meet the requirements of delay-sensitive Web applications (e.g., WebRTC). **Takeaways.** Collectively, our measurement study and analysis demonstrate that due to the unique LEO dynamics and lossy links in LSNs, end-to-end communications suffer from consistent, frequent, and highly-dynamic packet losses, which further affects the QoE of delay-sensitive applications (e.g., Web browsing and WebRTC). Existing fast recovery methods either cause extra waste of scarce bandwidth resources in LSNs, or are not applicable for LSNs with unique dynamic topology. The *status quo* motivates us to seek an effective solution to improve Web user experience in LSNs.

3 SATGUARD DESIGN AND IMPLEMENTATION

We present SATGUARD, a novel in-orbit loss recovery mechanism that can effectively improve user-perceived Web QoE.

3.1 System Overview

Architecture and baseline approach. Figure 7 plots a high-level view of SATGUARD upon today's LSN architecture. SATGUARD first adopts a baseline LL-ReTx approach to fast recover packet losses instead of requiring endpoint retransmission. The baseline LL-ReTx incorporates a *hop-specific* sequence number, called **HSeq**, to facilitate fast loss detection and local recovery. Without loss of generality, we define that packets are forwarded from an upstream node to its

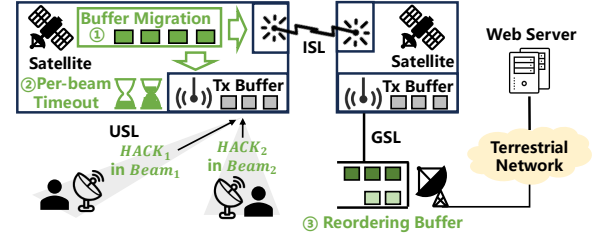


Figure 7: SATGUARD design overview, where green components are key techniques proposed in SATGUARD.

downstream node. LL-ReTx requires the upstream node to sequentially packs an HSeq in the header of each incoming packet and buffers a copy of this packet in the network interface (i.e., Tx buffer). On the downstream node, LL-ReTx checks the HSeq of arriving packets and sends a link-local feedback, denoted as **HACK**, which can instruct the upstream node to detect and retransmit lost packets. Basically, the HACK is sent after each packet arrives, which contains the information that a packet has been received. Once an HACK is received on the upstream node, the corresponding packet can be deleted from the buffer. A packet loss can be detected on the upstream node by examining the HACK mutation (e.g., when receiving a larger and discontinuous HACK). When a packet loss is detected, the upstream node can retransmit the corresponding packet from the buffer locally, instead of sender-side retransmissions.

Challenges caused by unique LSN characteristics. While at first glance, the above baseline approach can attain fast link-local retransmission, we highlight three challenges caused by the unique LSN characteristics that can hinder the effectiveness of LL-ReTx.

(1) Buffer invalidation due to LEO dynamics: Due to the LEO dynamics and endless ground-satellite handovers (i.e., link disconnection and reconstruction), the relationship between the upstream and downstream node is not stable, resulting in LL-ReTx buffer invalidation during handovers. For example, LL-ReTx may fail if the old upstream node with the buffered packets disconnects from the downstream node, and the new upstream node does not have the corresponding packets in its buffer.

(2) Inaccurate loss detection: The basic LL-ReTx mechanism works well for loss detection and recovery in *one-to-one* links, such as ISLs where one upstream satellite forwards packets to another downstream satellite. However, for *one-to-many* shared links like USLs where the upstream satellite can simultaneously forward packets to a number of downstream user terminals, HACKs from multiple downstream nodes might arrive at the upstream node out of order. In this situation, discontinuous HACK observed by the LL-ReTx upstream node does not necessarily indicate packet losses.

(3) Compatible with endpoint protocols: Even though lost packets can be recovered by LL-ReTx, the receiver might still receive out-of-order packets, which may mislead the protocol behaviors on endpoints and hinder the user-perceived QoE. For example, some endpoint mechanisms detect packet losses based on the order of received packets. While lost packets have been recovered by LL-ReTx at intermediate nodes, the consequential out-of-order packets observed by the endpoints can still trigger sender-side retransmission, which wastes scarce bandwidth resources in LSNs.

SATGUARD addresses the aforementioned challenges by three key techniques marked in green in Figure 7 upon the baseline approach.

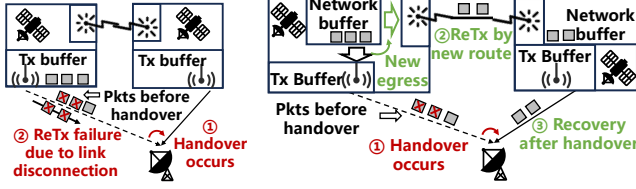


Figure 8: LL-ReTx failure due to dynamics.

Figure 9: Network-layer ReTx can reroute packets in a new path.

(1) **Handover-aware buffer migration (§3.2):** To tackle the buffer invalidation caused by LEO dynamics, SATGUARD exploits the predictable handover information from the LSN operator and routes buffered content (e.g., the lost packets) from the old upstream node to the new upstream node in time to avoid LL-ReTx failure.

(2) **Efficient loss detection via timer adaptation (§3.3):** To avoid inaccurate loss detection on one-to-many shared links, SATGUARD introduces a beam-specific timer adaptation method to efficiently detect user-side packet losses at fine granularity.

(3) **In-network order preserving (§3.4):** To mask out-of-order packets caused by LL-ReTx, SATGUARD preserves packet orders on ground facilities (e.g., satellite terminals and ground stations) before forwarding Web traffic to the user equipment (e.g., user laptops).

3.2 Handover-aware Seamless Buffer Migration

Network-layer buffer. Figure 8 shows a concrete example where the baseline LL-ReTx fails due to the LEO dynamics. Assume packets are forwarded from the satellite to the ground equipment (e.g., GS or dish). Satellites move from right to left in this example. After a handover, the old upstream satellite (i.e., the left one), which buffers packets, is no longer connected to the GS, resulting in a LL-ReTx failure. SATGUARD leverages a network-layer buffer to store packets temporarily. As shown in Figure 9, during the handover, the GS is disconnected from the old satellite (the left one) and then connected to the new satellite (the right one). SATGUARD exploits a handover-aware buffer migration strategy which perceives the handover and migrates the buffered packets from the old upstream node to the new upstream node correctly and timely.

How to find the new satellite? For SATGUARD's buffer migration, one critical problem is how the old satellite knows which new satellite the ground equipment will connect to. To solve this issue, SATGUARD exploits a key insight obtained from today's emerging LSN: operators typically leverage a centralized global scheduler to *pre-calculate* and decide ground-satellite connectivity in different time slots [59]. Hence, SATGUARD incorporates a connectivity cache on each satellite, which pre-caches the ground-satellite connectivity pre-calculated by the LSN operator. During a handover, the old satellite thus checks its connectivity cache to identify the new satellite that will serve the ground equipment after the handover.

When and how to migrate buffered contents? Once the new satellite (i.e., the migration destination) is decided, SATGUARD relies on space routing (e.g., [19]) to forward the buffered content to the new satellite. The underlying routing mechanism takes care of the route calculation, and updates the routing table once the handover happens. Considering that the migration of buffered content can also take a certain amount of time, SATGUARD does not start the migration just at the beginning of the handover. Instead, because the time point at which the handover occurs can be predictable

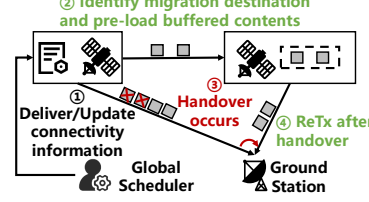


Figure 10: Ahead-of-handover

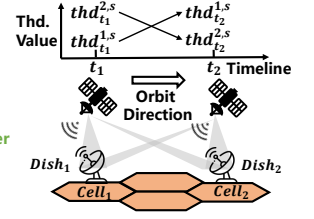


Figure 11: Beam-based timer setting and update.

according to the connectivity cache, SATGUARD starts to forward buffered content ahead of the handover, i.e., t seconds earlier than the handover time. SATGUARD sets t approximately to the ratio between buffer size and link capacity. Figure 10 shows an example of such ahead-of-handover migration. First, the LSN operator's global scheduler pre-calculates the connectivity which is pre-cached on satellites in advance. Second, just before the handover occurs, packets that are not acknowledged in the buffer are migrated to the new ingress satellite (i.e., the new upstream node). After the handover, packets can be retransmitted directly from the new satellite.

3.3 Beam-based Timer Adaptation

For one-to-many shared links (e.g., the radio USL on satellites that serve a number of land-based users), it is difficult to identify packet losses based on discontinuous HACK, since all the downstream nodes may send the HACKs back asynchronously. One approach to locally identify packet losses for LL-ReTx is to set a timer and treat unacknowledged packets after timeout as lost. For example, the LL-ReTx used in standard 802.11 WiFi [24] suggests setting a fixed timer on the WiFi access point to detect packet losses. However, unlike conventional WiFi which serves users in nearby locations, satellites have a significantly wider communication coverage. The distance between land-based users and satellites varies greatly, resulting in inherent propagation delay differences among users served by a satellite. A fixed timeout value can not effectively identify packet losses in USLs with dynamic time-varying propagation distances.

Beam-based timer adaptation. To effectively detect packet losses on USLs, SATGUARD exploits another key insight in LSNs: emerging satellites typically use multiple high-throughput spot-beams to serve geo-distributed users grouped into *cells*, which are terrestrial regions decided by the LSN operator. A cell clusters users in geo-graphically nearby locations. Therefore, SATGUARD classifies HACKs from geo-distributed users based on the beam the users are attached to, and sets an independent timer for each beam to detect lost packets. SATGUARD dynamically calculates $thd_t^{k,s}$, the timeout threshold for the cell attached to spot-beam k of satellite s in time t ,

and $thd_t^{k,s} = \frac{2 \times \text{distance}_{t, \text{cell}^{k,s}}}{c}$, where $\text{distance}_{t, \text{cell}^{k,s}}$ is the straight-line physical distance from the center of the k -attached cell (denoted as cell^k) to the satellite s in time t , and c is the propagation speed of electromagnetic waves. For each packet sent from s to users in cell cell^k in time t , if s does not receive HACK for $thd_t^{k,s}$ seconds, the packet is considered lost. Figure 11 plots an example of a beam-based timer setting and update. In time t_1 , the satellite s was closer to Dish_1 in Cell_1 , resulting in $thd_{t_1}^{1,s} < thd_{t_1}^{2,s}$. As the satellite moves, it gets closer to Dish_2 in Cell_2 , thus $thd_{t_2}^{1,s} > thd_{t_2}^{2,s}$.

3.4 In-network Order Preserving

SATGUARD adopts a ground-assisted order preserving buffer to migrate out-of-order packets caused by LL-ReTx. The buffer only needs to be deployed at the ground facilities (*i.e.*, ground stations or satellite terminals) and does not require modifications on endpoints. **Order identification.** SATGUARD adds a flow-level network-layer sequence number (FSeq) to packets before they enter the satellite network (*i.e.*, at the dish), and does not allow the FSeq to be modified by intermediate nodes, so the correct order can be recognized when they leave lossy satellite environment (*e.g.*, at the GS).

Reordering packets by FSeq. The basic operation of the buffer is to wait for local retransmitted packets appropriately and forward packets in order. Basically, the reordering buffer checks the FSeq of received packets. Once the FSeq mutation is detected, the packet will be buffered. The reordering buffer also discards duplicate packets based on duplicate FSeq. The reordering is executed at the flow-level, so there is no head-of-line blocking problem.

3.5 Overhead Optimization

Partial traffic processing. Processing HSeq/HACK and updating the header of packets on satellite routers can involve additional computation overhead. SATGUARD adopts a *partial traffic processing* mechanism, and only delay-sensitive traffic with stringent performance requirements will be processed by SATGUARD's local recovery mechanism. In practice, satellite Internet operators can mark the delay-sensitive traffic of high-value customers by configuring the Traffic Class option [14] in the standard IP header.

Dealing with ReTx and HACK loss. ReTx packets may also be lost. Some previous efforts (*e.g.*, [33]) retransmit a packet multiple times, which involves additional bandwidth overhead. To identify the loss of ReTx packets, SATGUARD embeds a larger HSeq instead of the original one to ReTx packets according to the latest sending order. Then, the loss of ReTx packets can also be detected by the mutation of HACK. The loss of HACK may cause unnecessary retransmission, which can be mitigated by enabling cumulative HACK. Since SATGUARD reassigns a new larger HSeq to a ReTx packet, the HACK in SATGUARD is designed to carry the information about the biggest interval of the most recently received consecutive HSeq.

3.6 SATGUARD Implementation

We have implemented a SATGUARD prototype with these key features described above. Since many emerging satellites run Linux-like on-board operating systems (*e.g.*, [17]), we implement the current version of SATGUARD upon Linux kernel 5.4.

Exploiting IPv6 extension to embed recovery information in packets. To facilitate compatible deployment, we implement SATGUARD's network-level HSeq, HACK, and FSeq based on the standard IPv6 Hop-by-Hop options header [14]. In particular, each HSeq, HACK and FSeq occupies 16 bits in one options header respectively. To implement the in-network functions of SATGUARD, we use `lib_netfilterqueue` [26, 43] to capture packets in kernel space for packet parsing and processing at intermediate nodes.

Leveraging segment routing to guide fast buffer migration. We implement SATGUARD's buffer migration described in §3.2 based on Segment Routing (SR) [18] which is intrinsically supported by Linux kernel. SR allows a network node to append a header to packets that contains a list of segments, which are instructions that can

be executed on subsequent nodes in the network. Therefore, when migrating buffered packets during a handover, SATGUARD leverages SR to re-route packets to the predicted next ingress satellite.

4 PERFORMANCE EVALUATION

4.1 Experiment Setup

We use a recent tool [37] to build a container-based LSN simulator based on real constellation information (*e.g.*, real orbital parameters and distribution of ground stations). The LSN simulator can mimic the LEO dynamics and network behaviors of large-scale LSNs. Each container is a Linux docker instance simulating a network node (*i.e.*, a satellite, ground station or satellite terminal). Moreover, our LSN simulator supports the run of real TCP/IP stack and thus we can load our SATGUARD implementation on each simulated network node and load interactive Web traffic in our experiments. We deploy the LSN simulator on two DELL R940xa rack servers and each machine has two Intel Xeon 5217 Processors and 8GB RDIMM. **Network configurations.** We use our LSN simulator to mimic the network environment of Starlink phase-I constellation which consists of 1584 satellites operating in 72 orbital planes. Further, we configure the packet loss rate in the experimental environment based on our real-world measurements collected from the operational Starlink. As LEO satellites move, we simulate the ground-satellite handovers, and the ground equipment selects a new ingress satellite with the shortest distance [11, 59]. This strategy is widely used in today's satellite communication systems. The space segment of the LSN adopts the +Grid connectivity [22], which indicates that each satellite connects to two adjacent satellites in the same orbit (front and rear), and other two satellites in adjacent orbits (left and right). The maximum capacity of the ISL and GSL is configured to 1Gbps [6, 10]. Routing for LSNs is not the focus of this paper, and we use the well-known snapshot routing [19] as the underlying routing mechanism. In snapshot routing, each network node has a pre-calculated valid routing table for each time slot, which indicates how to forward incoming packets on network nodes.

Approaches of comparison. We compared the performance of SATGUARD with recent state-of-the-art mechanisms for improving network performance in lossy environment: SaTCP [7], Tambur [53], M-PEP [8], and LinkGuard [33]. SaTCP is a link-layer-assisted transport enhancement mechanism deployed in the end points in emerging LSNs. SaTCP leverages the handover signal to freeze the congestion window to keep high throughput during handover. The key idea behind Tambur is to leverage streaming codes [3] to guide receiver to recover lost data when there is a guard space where all packets are received. M-PEP is a solution based on the well-studied Performance Enhancement Proxy (PEP) technology, which uses in-network proxies to transparently split TCP connections and mitigate the impact of long delay and high loss. We deploy M-PEPs on the ground stations, dishes and their ingress satellites. LinkGuard [33] uses a small buffer to achieve link-local retransmission without per-flow state to reduce tail delay in links caused by packet losses. Next we verify the effectiveness of SATGUARD from both the network and application perspectives.

4.2 Network-level Performance

Improvement on tail delay. Figure 12 plots the 99th percentile delay for different communication pairs among New York City (NYC),

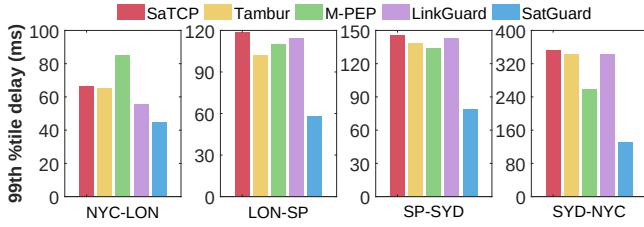


Figure 12: 99th percentile user-perceived end-to-end delay of different communication pairs.

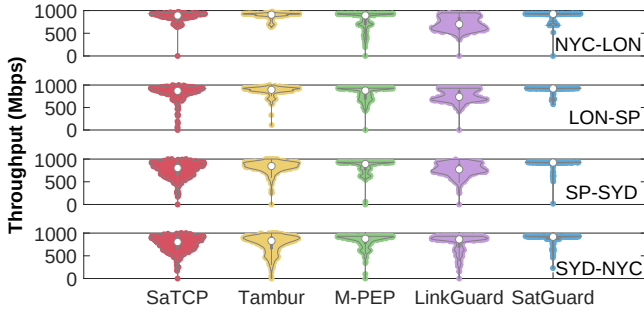


Figure 13: Violin plot of the achievable end-to-end throughput between different communication pairs.

London (LON), Sydney (SYD) and Sao Paulo (SP). Specifically, the propagation delay for the paths of NYC-LON, LON-SP, SP-SYD, and SYD-NYC ranges from 10.4 to 26.4 ms, 17.2 to 35.1 ms, 45.3 to 59.4 ms, and 44.5 to 114.4 ms, respectively. Note that the variation in delay results from changes in transmission paths due to satellite handovers. Since SaTCP only freezes congestion window of the sender during handovers, it can not accelerate the loss recovery and it retransmits lost packets from the sender, resulting in high tail delay up to 351.97 ms for SYD-NYC communication. Although Tambur leverages streaming codes to guide local recovery on the receiver, the preset guard spaces in LSNs can not be guaranteed due to consistent packet losses in an error-prone environment, which affects the success rate of local recovery and prolongs the delay up to 342.27 ms. The tail delay in M-PEP ranges from 84.98 ms to 258.10 ms, because M-PEP needs to migrate various state information (e.g., the parameters related to rate control) from the previous ingress satellite to the new ingress satellite before restarting the transmission, which can cost as much as 91.2 ms, depending on the transmission path between the two satellites. After handovers, the buffered packets in the LinkGuard LL-ReTx buffer can not be retransmitted successfully, triggering long-time sender-based recovery and leading to a tail delay up to 342.90 ms. SATGUARD leverages network-layer buffer and route updates information to achieve fast buffer migration after handovers, which effectively reduces the tail delay by up to 56.2% for all communication pairs.

Improvement on end-to-end throughput. As shown in Figure 13, SaTCP freezes the congestion window during the handover by predicting the moment when handovers occur. However, packet losses due to interference factors such as bad weather are unpredictable, which makes the throughput in SaTCP less than 500 Mbps in 13.69% of the time. In Tambur, once any packet loss occurs during the preset guard spaces, lost packets cannot be recovered in the receiver, thus reducing the sending rate to less than 500 Mbps in 12.94% of the time. During handovers, M-PEP needs to resume the

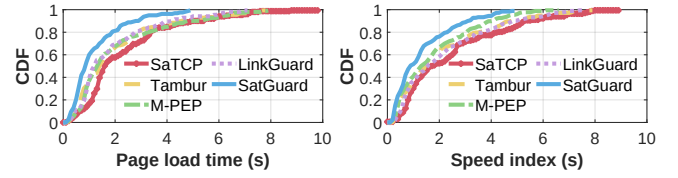


Figure 14: The CDF of page load time and speed index for browsing websites by different loss recovery approaches.

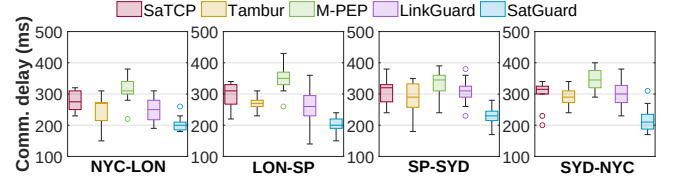


Figure 15: User-perceived one-way WebRTC session delay. transmission after state migration, so the transmission state frequently goes through the slow start phase. For LinkGuard, although it can effectively recovery packet losses in non-handover scenarios, lost packets caused by LEO dynamics are still perceived by the sender. Besides, due to unreasonable HACK timeout settings in the one-to-many shared links, the loss detection efficiency is reduced, so the attainable average throughput varies from 719.3 Mbps to 785.2 Mbps. Through integrating handover-aware buffer migration and HACK timeout adaptation to extend LL-ReTx, SATGUARD can locally recover lost packets in various scenarios, maintaining high throughput of more than 800 Mbps for 83.04% of the time.

4.3 Application-level Performance

SATGUARD can accelerate Web browsing in LSNs. We use a tool mahimahi [48] to record and replay the workload of browsing all sites in the Alexa Top 200 [16] with different reliability control mechanism. As shown in Figure 14, SATGUARD achieves the lowest page load time and speed index. Specifically, the average page load time in SATGUARD is 48.3%, 38.1%, 40.2% and 34.4% shorter than that in SaTCP, Tambur, M-PEP, and LinkGuard respectively. The achieved speed index in SATGUARD is 46.6%/36.2%/25.9% and 38.9% shorter than that in other mechanisms, respectively. The main reason is that SATGUARD can recover various losses in time, while (i) SaTCP has to re-request resource from Web servers when any loss occurs, (ii) Tambur can not set appropriate redundancy in such a short flow, (iii) M-PEP has to load resources until state migration completion and transport connection reconstruction, and (iv) LinkGuard behaves similarly to SaTCP in the face of packet losses due to handovers.

SATGUARD can sustain low-delay and smooth WebRTC communications. Finally, to evaluate the performance improvement of SATGUARD on Web-based RTC applications, we use two ThinkPad X1 laptops to connect to our LSN simulator as the sender and receiver, and build WebRTC sessions via kurento [25]. We run each RTC session for at least 1 hour and use the WebRTC statistic APIs to capture core QoE metrics including delay, jitter and FPS.

(i) *End-to-end user-perceived communication delay.* Figure 15 plots the communication delay of different sessions. The average communication delay in SATGUARD is 213.3 ms, and the delay variation among different pairs is small, because most lost packets can be recovered locally without passing through long-path retransmission. Therefore, the communication delay in M-PEP also fluctuates very

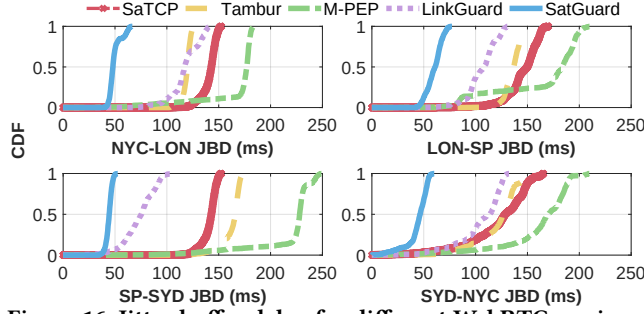


Figure 16: Jitter buffer delay for different WebRTC sessions.

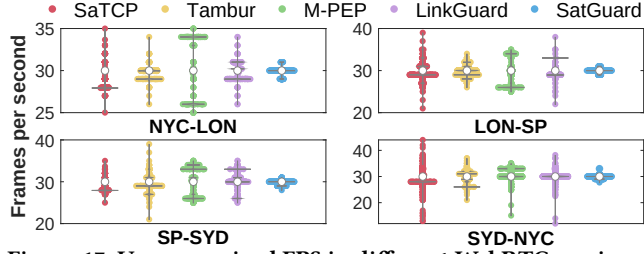


Figure 17: User-perceived FPS in different WebRTC sessions.

little. However, due to long-term state migration and connection reconstruction during every handover, the communication delay is stable at a high value of about 335.8 ms. Due to the lack of local recovery capacity to dynamic packet losses, SaTCP and LinkGuard, in such cases, must endure the impact of sender-based retransmission on the communication delay, which was aggravated with the increase of the communication distance, resulting in an average delay of 298.6 ms and 279.4 ms. The average delay in Tambur is about 267.2 ms because the frames can be lost in its guard space. (ii) *Jitter buffer delay (JBD)*. As shown in Figure 16, the JBD in M-PEP is more than 150 ms in up to 94.1% of the session period, because frequent long-term *state migration* causes the receiver to experience large jitter. Tambur and SaTCP cause JBD higher than 100 ms in at least 86.2% and 84.3% of the time because they all need to retransmit lost or unrecoverable packets from the sender. Although LinkGuard can achieve low jitter in static scenarios constantly by local retransmission, it fails in dynamic scenarios, causing that the JBD fluctuates greatly, from 52.1 to 130.2 ms. The JBD in SATGUARD is less than 73.1 ms because the packet losses caused by handovers can also be effectively recovered by *buffer migration*.

(iii) *Frames per second (FPS)*. As plotted in Figure 17, the FPS of M-PEP is stable at about 33 and 26 most of the time. In fact, we find that the FPS of M-PEP is hopping between the two values frequently, mainly because during the state migration among PEP nodes, a large amount of accumulated frames are played quickly after the transmission is resumed. The FPS in SaTCP is less than 29.5 in 41.6% of time and varies from 12.3 to 43.2. Such fluctuation is mainly because lost frames can not being recovered in time and received frames wait too long in the jitter buffer. Although the average FPS achieved by Tambur and LinkGuard are 29.91 and 29.83 respectively, the standard deviation of Tambur and LinkGuard is 423% and 336% higher than that of SATGUARD, indicating an unexpected FPS variation which may hinder user experience. The main reason, similar to SaTCP, is that the lost frame information can not be recovered by LinkGuard during handovers and by Tambur when loss occurs in preset guard space, and thus discarded in playout.

Empowered by handover-aware LL-ReTx, our SATGUARD keeps a stable 30 FPS and guarantees smooth experience for WebRTC.

5 RELATED WORK

Real-world LSN measurement. Authors in [47] measure the performance of QUIC [38] and TCP in Starlink and find QUIC can achieve more stable throughput for uploads but has a lower throughput for downloads. Measurements in [34] show frequent and severe packet losses of up to 50% of packets, which usually occurs during handovers in Starlink. Authors in [59] reveal the existence of Starlink’s global scheduler through long-duration measurements, which allocates satellites to terminals every 15 seconds. In [49], authors show the composition of the Starlink access and backbone network through a large number of traceroute results. [21] and [60] observes throughput and delay fluctuations every 15 seconds respectively in the current Starlink network. These explorations demystify the network architecture and protocols of emerging LSNs. Our measurement study in this paper goes one step further, by characterizing the QoE of representative Web services.

Web application performance and optimization in lossy environment. Authors in [52] find that in a wireless environment, inter-Radio-Access-Technology handover caused by user mobility is the most important reason for QoE degradation. WiseTrans [65] is a selection mechanism to adaptively change the transport protocol that best suits the current network environment to reduce delay tail and request completion time, which can be used in collaboration with SATGUARD to reduce the impact of switching transport protocols. Authors in [15] design a WebRTC-compliant video conferencing platform that leverages the larger bandwidth provided by multipath to meet the rising demand for high-resolution video conferencing. SATGUARD can improve transmission performance on each subpath, further improving overall performance.

6 CONCLUSION

This paper proposes SATGUARD, a distributed in-orbit loss recovery mechanism based on link-local retransmission. Key ideas behind SATGUARD include (i) exploiting handover information provided by satellite operators to effectively migrate buffers to correct node to prevent buffer invalidation, (ii) leveraging dynamic beam-specific timeout calculation to set the appropriate loss detection parameters for users in different cells to improve recovery efficiency, and (iii) processing out-of-order packets in the ground to make in-network recovery transparent to the upper-layer protocols and applications. The implementation of SATGUARD is based on existing standard protocols, and it is compatible with existing Internet protocol stack. Extensive trace-driven evaluations show that SATGUARD can reduce average page load time for Web browsing and user-perceived communication delay for WebRTC by up to 48.3% and 57.4%. The source code of SATGUARD can be found in <https://doi.org/10.5281/zenodo.10675228> and [40].

ACKNOWLEDGMENTS

This work was supported by the National Key R&D Program of China (No. 2022YFB3105202) and National Natural Science Foundation of China (NSFC No. 62372259 and No. 62132004). Hewu Li and Zeqi Lai are the corresponding authors. Thanks to our partners for deploying the Starlink network measurement environment.

REFERENCES

- [1] [n. d.]. Demystifying page load performance with WProf, author=Wang, Xiao Sophia and Balasubramanian, Aruna and Krishnamurthy, Arvind and Wetherall, David, booktitle=10th USENIX Symposium on Networked Systems Design and Implementation (NSDI 13), pages=473–485, year=2013.
- [2] Muhammad Ayaz, Azween Abdullah, and Ibrahim Faye. 2010. Hop-by-hop reliable data deliveries for underwater wireless sensor networks. In *2010 International Conference on Broadband, Wireless Computing, Communication and Applications*. IEEE, 363–368.
- [3] Ahmed Badr, Pratik Patil, Ashish Khisti, Wai-Tian Tan, and John Apostolopoulos. 2016. Layered constructions for low-delay streaming codes. *IEEE Transactions on Information Theory* 63, 1 (2016), 111–141.
- [4] John Border, Markku Kojo, Jim Griner, Gabriel Montenegro, and Zach Shelby. 2001. *Performance enhancing proxies intended to mitigate link-related degradations*. Technical Report.
- [5] Alan Boyle. 2019. Amazon to offer broadband access from orbit with 3,236-satellite 'Project Kuiper' constellation. <https://www.geekwire.com/2019/amazon-project-kuiper-broadband-satellite/>. [Online; accessed 20-Feb-2024].
- [6] Jiaqi Cao, Shengli Zhang, Qingxia Chen, Houtian Wang, Mingzhe Wang, and Naijin Liu. 2023. Computing-Aware Routing for LEO Satellite Networks: A Transmission and Computation Integration Approach. *IEEE Transactions on Vehicular Technology* (2023).
- [7] Xuyang Cao and Xinyu Zhang. 2023. SaTCP: Link-Layer Informed TCP Adaptation for Highly Dynamic LEO Satellite Networks. In *IEEE INFOCOM 2023-IEEE Conference on Computer Communications*. IEEE, 1–10.
- [8] Alberto Caponi, Andrea Detti, Michele Luglio, Cesare Roseti, and Francesco Zampognaro. 2015. Mobile-PEP: Satellite terminal handover preserving service continuity. In *2015 International Symposium on Wireless Communication Systems (ISWCS)*. IEEE, 221–225.
- [9] Neal Cardwell, Yuchung Cheng, C. Stephen Gunn, Soheil Hassas Yeganeh, and Van Jacobson. 2016. BBR: Congestion-Based Congestion Control: Measuring Bottleneck Bandwidth and Round-Trip Propagation Time. *Queue* 14, 5 (oct 2016), 20–53. <https://doi.org/10.1145/3012426.3022184>
- [10] Quan Chen, Jianming Guo, Lei Yang, Xianfeng Liu, and Xiaoqian Chen. 2019. Topology virtualization and dynamics shielding method for LEO satellite networks. *IEEE Communications Letters* 24, 2 (2019), 433–437.
- [11] Pulak K Chowdhury, Mohammed Atiqzaman, and William Ivancic. 2006. Handover schemes in satellite networks: state-of-the-art and future research directions. *IEEE Communications Surveys & Tutorials* 8, 4 (2006), 2–14.
- [12] Federal Communications Commission. 2024. SpaceX Non-geostationary Satellite System. https://licensing.fcc.gov/myibfs/web/userHome.do?attachment_key=1158350. [Online; accessed 20-Feb-2024].
- [13] Telesat Company. 2023. Telesat. <https://www.telesat.com/>. [Online; accessed 20-Feb-2024].
- [14] Steve Deering and Robert Hinden. 2017. RFC 8200: Internet protocol, version 6 (IPv6) specification.
- [15] Sandesh Dhawaskar Sathyanarayana, Kyunghan Lee, Dirk Grunwald, and Sangtae Ha. 2023. Converge: QoE-driven Multipath Video Conferencing over WebRTC. In *Proceedings of the ACM SIGCOMM 2023 Conference*. 637–653.
- [16] Expired Domain Name Search Engine. 2024. Alexa Top 500 Websites. <https://www.expireddomains.net/alexa-top-websites/>. [Online; accessed 20-Feb-2024].
- [17] Felgo. 2022. Embedded operating systems. <https://blog.felgo.com/embedded/embedded-operating-systems>. [Online; accessed 20-Feb-2024].
- [18] Clarence Filsfils, Stefano Previdi, Les Ginsberg, Bruno Decraene, Stephane Litkowski, and Rob Shakir. 2018. *Segment routing architecture*. Technical Report.
- [19] Daniel Fischer, David Basin, Knut Eckstein, and Thomas Engel. 2012. Predictable mobile routing for spacecraft networks. *IEEE Transactions on Mobile Computing* 12, 6 (2012), 1174–1187.
- [20] Silas L Fong, Salma Emara, Baochun Li, Ashish Khisti, Wai-Tian Tan, Xiaoqing Zhu, and John Apostolopoulos. 2019. Low-latency network-adaptive error control for interactive streaming. In *Proceedings of the 27th ACM International Conference on Multimedia*. 438–446.
- [21] Johan Garcia, Simon Sundberg, Giuseppe Caso, and Anna Brunstrom. 2023. Multi-Timescale Evaluation of Starlink Throughput. In *Proceedings of the 1st ACM Workshop on LEO Networking and Communication*. 31–36.
- [22] Giacomo Giuliani, Tommaso Ciussani, Adrian Perrig, and Ankit Singla. 2021. ICARUS: Attacking low earth orbit satellite networks. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. 317–331.
- [23] Browsertime Group. 2024. Browsertime. <https://www.sitespeed.io/documentation/browsertime/introduction/>. [Online; accessed 20-Feb-2024].
- [24] IEEE 802.11 Working Group. 2024. IEEE 802.11T Wireless Local Area Networks. <https://www.ieee802.org/11/>. [Online; accessed 20-Feb-2024].
- [25] Kurento Group. 2024. Kurento. <https://github.com/Kurento>. [Online; accessed 20-Feb-2024].
- [26] Netfilter Group. 2023. Netfilter. https://www.netfilter.org/projects/libnetfilter_queue/index.html. [Online; accessed 20-Feb-2024].
- [27] WebRTC Group. 2024. WebRTC. <https://webrtc.org/>. [Online; accessed 20-Feb-2024].
- [28] Sangtae Ha, Injong Rhee, and Lisong Xu. 2008. CUBIC: a new TCP-friendly high-speed TCP variant. *ACM SIGOPS operating systems review* 42, 5 (2008), 64–74.
- [29] Mark Handley. 2018. Delay is Not an Option: Low Latency Routing in Space. In *Proceedings of the 17th ACM Workshop on Hot Topics in Networks (HotNets)* (Redmond, WA, USA). Association for Computing Machinery, New York, NY, USA, 85–91. <https://doi.org/10.1145/3286062.3286075>
- [30] Han Hu, Sheng Cheng, Xinggong Zhang, and Zongming Guo. 2021. LightFEC: Network adaptive FEC with a lightweight deep-learning approach. In *Proceedings of the 29th ACM International Conference on Multimedia*. 3592–3600.
- [31] M Hussein and A Hanani. 2016. Routing in IP/LEO satellite communication systems: past, present and future. *Int J Electron Commun Eng* 3 (2016), 745.
- [32] ITU. 2018. F.743.6: Service requirements for next generation content delivery networks. <https://www.itu.int/rec/T-REC-F.743.6-201808-I>. [Online; accessed 20-Feb-2024].
- [33] Raj Joshi, Cha Hwan Song, Xin Zhe Khoi, Nishant Budhdev, Ayush Mishra, Mun Choon Chan, and Ben Leong. 2023. Masking Corruption Packet Losses in Datacenter Networks with Link-local Retransmission. In *Proceedings of the ACM SIGCOMM 2023 Conference*. 288–304.
- [34] Mohamed M. Kassem, Aravindh Raman, Diego Perino, and Nishanth Sastry. 2022. A Browser-Side View of Starlink Connectivity. In *Proceedings of the 22nd ACM Internet Measurement Conference (Nice, France) (IMC '22)*. Association for Computing Machinery, New York, NY, USA, 151–158. <https://doi.org/10.1145/3517745.3561457>
- [35] Simon Kassing, Debopam Bhattacharjee, André Baptista Águas, Jens Eirik Saethre, and Ankit Singla. 2020. Exploring the "Internet from Space" with Hypatia. In *Proceedings of the ACM Internet Measurement Conference (Virtual Event, USA) (IMC '20)*. Association for Computing Machinery, New York, NY, USA, 214–229. <https://doi.org/10.1145/3419394.3423635>
- [36] Inkyu Kim, Jaekuk Baek, Sangik Han, and Youngnam Han. 2020. The performance analysis of multi-hop relay DTN communication system in interplanetary network. In *2020 International Conference on Information and Communication Technology Convergence (ICTC)*. IEEE, 1136–1141.
- [37] Zeqi Lai, Hewu Li, Yangtao Deng, Qian Wu, Jun Liu, Yuanjie Li, Jihao Li, Lixin Liu, Weisen Liu, and Jianping Wu. 2023. StarryNet: Empowering Researchers to Evaluate Futuristic Integrated Space and Terrestrial Networks. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. 1309–1324.
- [38] Adam Langley, Alistair Riddoch, Alyssa Wilk, Antonio Vicente, Charles Krasnic, Dan Zhang, Fan Yang, Fedor Kouranov, Ian Swett, Janardhan Iyengar, et al. 2017. The quic transport protocol: Design and internet-scale deployment. In *Proceedings of the conference of the ACM special interest group on data communication*. 183–196.
- [39] Insoo Lee, Seyeon Kim, Sandesh Sathyanarayana, Kyungmin Bin, Song Chong, Kyunghan Lee, Dirk Grunwald, and Sangtae Ha. 2022. R-FEC: RL-based FEC Adjustment for Better QoE in WebRTC. In *Proceedings of the 30th ACM International Conference on Multimedia*. 2948–2956.
- [40] Jihao Li. 2024. SatGuard Implementation. <https://github.com/SpaceNetLab/SatGuard>. [Online; accessed 20-Feb-2024].
- [41] Ming Li, Devesh Agrawal, Deepak Ganesan, Arun Venkataramani, and Himanshu Agrawal. 2009. Block-switched Networks: A New Paradigm for Wireless Transport.. In *NSDI*, Vol. 9. 423–436.
- [42] Lixin Liu, Hewu Li, Yuanjie Li, Zeqi Lai, Yangtao Deng, Yimei Chen, Wei Liu, and Qian Wu. 2022. Geographic Low-Earth-Orbit Networking without QoS Bottlenecks from Infrastructure Mobility. In *2022 IEEE/ACM 30th International Symposium on Quality of Service (IWQoS)*. IEEE, 1–10.
- [43] Jesus Llorente. 2018. Netfilter Extension. https://github.com/jlloriente/helpers_n_wrappers/blob/master/nfqueue3.py. [Online; accessed 20-Feb-2024].
- [44] Yan Lou, Yi Wu Zhao, Chunyi Chen, Shoufeng Tong, and Cheng Han. 2016. Analysis of Sun Outages Influence on GEO to LEO Communication. In *Wireless Communications, Networking and Applications*. Springer, 1017–1025.
- [45] Sami Ma, Yi Ching Chou, Haoyuan Zhao, Long Chen, Xiaoqiang Ma, and Jiangchuan Liu. 2023. Network Characteristics of LEO Satellite Constellations: A Starlink-Based Measurement from End Users. In *IEEE INFOCOM 2023-IEEE Conference on Computer Communications*. IEEE, 1–10.
- [46] François Michel, Quentin De Coninck, and Olivier Bonaventure. 2019. QUIC-FEC: Bringing the benefits of Forward Erasure Correction to QUIC. In *2019 IFIP Networking Conference (IFIP Networking)*. IEEE, 1–9.
- [47] François Michel, Martino Trevisan, Danilo Giordano, and Olivier Bonaventure. 2022. A first look at starlink performance. In *Proceedings of the 22nd ACM Internet Measurement Conference*. 130–136.
- [48] Ravi Netravali, Anirudh Sivaraman, Somak Das, Ameesh Goyal, Keith Winstein, James Mickens, and Hari Balakrishnan. 2015. Mahimahi: Accurate Record-and-Replay for HTTP.. In *Usenix annual technical conference*. 417–429.
- [49] Jianping Pan, Jinwei Zhao, and Lin Cai. 2023. Measuring a low-earth-orbit satellite network. *arXiv preprint arXiv:2307.06863* (2023).
- [50] JC Pavur, Martin Strohmeier, Vincent Lenders, and Ivan Martinovic. 2021. QPEP: An actionable approach to secure and performant broadband from geostationary

- orbit. (2021).
- [51] Ting Qu, Raj Joshi, Mun Choon Chan, Ben Leong, Deke Guo, and Zhong Liu. 2019. SQR: In-network packet loss recovery from link failures for highly reliable datacenter networks. In *2019 IEEE 27th International Conference on Network Protocols (ICNP)*. IEEE, 1–12.
 - [52] Mohammad Rajiullah, Andra Lutu, Ali Safari Khatouni, Mah-Rukh Fida, Marco Mellia, Anna Brunstrom, Ozgu Alay, Stefan Alfredsson, and Vincenzo Mancuso. 2019. Web experience in mobile networks: Lessons from two million page visits. In *The world wide web conference*. 1532–1543.
 - [53] Michael Rudow, Francis Y Yan, Abhishek Kumar, Ganesh Ananthanarayanan, Martin Ellis, and KV Rashmi. 2023. Tambur: Efficient loss recovery for videoconferencing via streaming codes. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. 953–971.
 - [54] SpaceX. 2024. The flat high-performance satellite terminal of Starlink. <https://www.starlink.com/specifications?spec=3>. [Online; accessed 20-Feb-2024].
 - [55] SpaceX. 2024. The high-performance satellite terminal of Starlink. <https://www.starlink.com/specifications?spec=2>. [Online; accessed 20-Feb-2024].
 - [56] SpaceX. 2024. The standard satellite terminal of Starlink. <https://www.starlink.com/specifications?spec=1>. [Online; accessed 20-Feb-2024].
 - [57] SpaceX. 2024. StarLink. <https://www.starlink.com/>. [Online; accessed 20-Feb-2024].
 - [58] Amazon Staff. 2023. The satellite terminal of Kuiper. <https://www.aboutamazon.com/news/innovation-at-amazon/heres-your-first-look-at-project-kuipers-low-cost-customer-terminals>. [Online; accessed 20-Feb-2024].
 - [59] Hammas Bin Tanveer, Mike Puchol, Rachee Singh, Antonio Bianchi, and Rishab Nithyanand. 2023. Making Sense of Constellations: Methodologies for Understanding Starlink's Scheduling Algorithms. *arXiv preprint arXiv:2307.00402* (2023).
 - [60] Pouria Tolouei. 2023. Analysing the Performance of Cloud Gaming over a Low-Earth Orbit Satellite Network. (2023).
 - [61] TS2. 2023. SpaceX's Starlink Surpasses 2 Million Active Customers Globally. <https://ts2.space/en/spacexs-starlink-surpasses-2-million-active-customers-globally/>. [Online; accessed 20-Feb-2024].
 - [62] W3C. 2014. Identifiers for WebRTC's Statistics API. <https://www.w3.org/TR/webrtc-stats/>. [Online; accessed 20-Feb-2024].
 - [63] Kinza Yasar. 2022. Starlink. <https://www.techtarget.com/whatis/definition/Starlink>. [Online; accessed 20-Feb-2024].
 - [64] Ramish Zafar. 2020. SpaceX Successfully Tests Inter-Satellite Starlink Connectivity Via Lasers. <https://wccfttech.com/spacex-starlink-satellite-laser-test/>. [Online; accessed 20-Feb-2024].
 - [65] Jia Zhang, Enhuan Dong, Zili Meng, Yuan Yang, Mingwei Xu, Sijie Yang, Miao Zhang, and Yang Yue. 2021. WiseTrans: Adaptive transport protocol selection for mobile web service. In *Proceedings of the Web Conference 2021*. 284–294.
 - [66] Mihail Zverev, Pablo Garrido, Fatima Fernandez, Josu Bilbao, Özgü Alay, Simone Ferlin, Anna Brunstrom, and Ramón Agüero. 2021. Robust QUIC: Integrating Practical Coding in a Low Latency Transport Protocol. *IEEE Access* 9 (2021), 138225–138244.

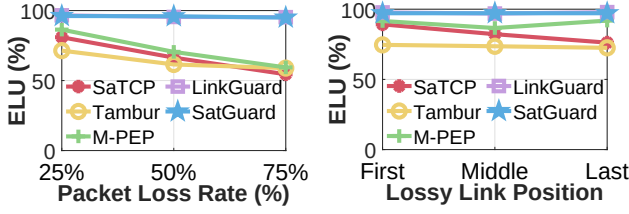


Figure 18: The improvement on ELU as the packet loss rate of links increases.

Figure 19: The improvement on ELU with losses in different positions.

CCA	Mechanism	Link Capacity= 800/900/1000 Mbps		
CUBIC [28]	w/o SATGUARD	748 Mbps	841 Mbps	934 Mbps
	w/ SATGUARD	748 Mbps	840 Mbps	923 Mbps
BBR [9]	w/o SATGUARD	747 Mbps	840 Mbps	934 Mbps
	w/ SATGUARD	747 Mbps	840 Mbps	905 Mbps

Table 1: Maximum attainable throughput w/o and w/ SATGUARD in a lossless environment while using different CCAs.

A OVERHEAD ANALYSIS

A.1 Overhead of Packet Reordering

Reordering delay. Note that SATGUARD's in-network packet reordering mechanism only involves additional delay when some packets are out-of-order. In our experimental environment, the average additional delay is 5.5% of the minimum one-way delay (OWD) for the communication pair. For example, the additional delay is about 2.75ms on average if the minimum OWD is about 50ms. Although the packet reordering mechanism slightly increases the average delay, it also significantly reduces the tail delay to sustain a smooth user experience as we have demonstrated in §4.

State maintenance overhead. The state required by SATGUARD's reordering buffer for each flow only contains: (a) the expected flow-level network-layer sequence number (FSeq) and (b) the current reordering time, which only consumes 8 additional bytes in total for each flow, suggesting that the overhead of maintaining the state for each flow is acceptable in practice. Once the received FSeq is expected or the reordering time exceeds the threshold, buffered packets will be forwarded and the state will be refreshed.

A.2 Packet Processing Overhead

Bandwidth consumption. In our current prototype implementation, SATGUARD's in-network packet processing mechanism involves only 0.7% extra bandwidth consumption. Specifically, the additional sequence number introduced by SATGUARD takes up 8 bytes in the header of each packet.

Computation overhead. Table 1 shows the reduction of maximum attainable throughput by enabling SATGUARD in a lossless

environment while using different congestion control algorithms (CCAs). In our current SATGUARD implementation, processing the additional fields at intermediate nodes does not affect the end-to-end throughput when the application traffic is less than 900 Mbps. Even when the link capacity is 1000 Mbps, SATGUARD only causes a 1.2-3.1% throughput reduction.

B EFFECTIVE LINK UTILIZATION ANALYSIS

Effective Link Utilization (ELU) captures how link resources are efficiently used to forward data packets. Specifically, the *ELU* of a packet p is calculated by $ELU_p = \frac{size_p^{raw} \times \#_hop_p^{lossless}}{size_p \times \#_hop_p}$. $size_p^{raw}$ is the original size of p , while $size_p$ refers to the real packet size transferred. Note that some protocols leverage network coding [20] to add redundancy in packets. In such situations we have $size_p^{raw} \leq size_p$. Further, $\#_hop_p^{lossless}$ and $\#_hop_p$ indicate the number of hops the packet has traveled, in an ideal lossless and a real lossy environment respectively. We use a simple example to explain the calculation of *ELU*. Assume there is a 2-hop path, denoted as ($S \rightarrow M \rightarrow R$), and ideally we have $\#_hop_p^{lossless} = 2$. A packet p from S to R is lost in node M in the first transmission, but it is successfully retransferred to R in the first retransmission. Then we have $\#_hop_p = 3$. If no redundancy is added to p , and $size_p^{raw} = size_p$, we can calculate $ELU_p = \frac{2}{3} = 67\%$. Intuitively, a higher *ELU* indicates that the network resources are used more *efficiently*.

Next, we evaluate the effective link utilization (ELU) achieved by different approaches under different packet loss scenarios. To simulate a lossy LSN environment, we change the packet loss rate of links. Figure 18 plots the ELU as the packet loss rate increases. The ELU of SaTCP and M-PEP drops dramatically from 80.9% to 54.5% because more and more lost packets need to be transmitted from the sender or the upload PEP node. Tambur has to increase the redundancy of information coding to improve the probability of local recovery in the receiver, resulting in a low ELU from 71.4% to 59.8%. LinkGuard and SATGUARD can maintain a high ELU above 94.01% in all scenarios. As SATGUARD exploits cache migration to efficiently conduct in-network loss recovery even in handovers, it achieves the highest ELU as compared to other approaches.

Figure 19 plots the ELU when packets are lost in the first, middle, and last hop of the entire path. SATGUARD achieves about 97.1% ELU in all scenarios. LinkGuard attains an approximate similar ELU as compared with SATGUARD. SaTCP, Tambur, and M-PEP cause much lower ELU (from 89%, 66%, and 91% to 76%, 62%, and 87%) since they retransmit packets from the sender or the PEP node near the sender. In particular, Tambur also transfers additional network coding information, which further increases the required bandwidth consumption. Besides, the closer to the receiver when packets are lost, the lower the ELU achieved by SaTCP.